Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

ОТЧЁТ

по лабораторной работе № 6
«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Выполнил студент: Иванов Виктор Сергеевич
Факультет прикладной информатики, группа K3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа «Мобильные и сетевые технологии»
Преподаватель: Говорова М. М.

Санкт-Петербург
2025

Цель

Изучить основные операции в СУБД MongoDB, включая создание базы данных, коллекций, вставку и изменение документов, а также базовую работу с индексами и запросами.

Ход работы

1. Установка и запуск MongoDB

MongoDB был установлен через Homebrew командой:

```
brew install mongodb-community
brew services start mongodb/brew/mongodb-community
```

Проверка подключения через оболочку:

```
mongosh
```

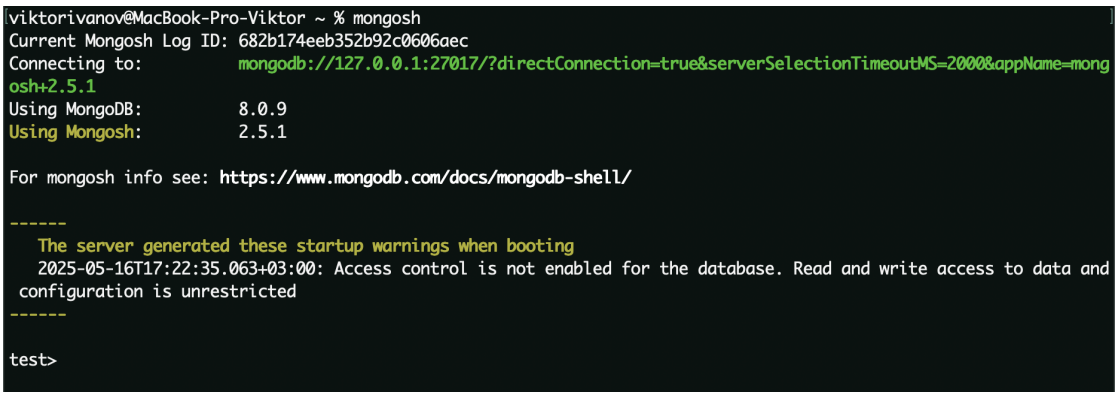


Рис. 1: Успешный запуск mongosh

2. Создание базы данных и коллекции

```
use learn
db.unicorns.insert({...})
...
```



Рис. 2: Заполнение коллекции

Вставка документа (второй способ)

```
document = ({
  name: "Dunx",
  loves: ["grape", "watermelon"],
  weight: 704,
  gender: "m",
  vampires: 165
})
db.unicorns.insert(document)
```



Рис. 3: Заполнение коллекции

2.2 Выборка данных

```
db.unicorns.find()
```

2.2.1 Выборка документов по полу и предпочтениям

```
// Самцы (отсортировано по имени)
db.unicorns.find({gender: "m"}).sort({name: 1})
```



Рис. 4: Пример выборки

```
// Самки (только первые 3 по имени)
db.unicorns.find({gender: "f"}).sort({name: 1}).limit(3)

// Самка, любящая carrot (первый найденный документ)
db.unicorns.findOne({gender: "f", loves: "carrot"})

// Альтернатива через find + limit
db.unicorns.find({gender: "f", loves: "carrot"}).limit(1)
```

2.2.2 Исключение полей из выборки

```
// Самцы, без loves и gender
db.unicorns.find(
  { gender: "m" },
```



Рис. 5: Самцы (отсортировано по имени)

```
  { loves: 0, gender: 0 }
)
```

2.2.3 Обратный порядок добавления

```
db.unicorns.find().sort({$natural: -1})
```

2.2.4 Вывод имени и первого предпочтения

```
db.unicorns.find(
  {},
  {
    _id: 0,
    name: 1,
    loves: { $slice: 1 }
  }
)
```



Рис. 6: Самки (только первые 3 по имени)



Рис. 7: Самка, любящая carrot (первый найденный документ)

2.3.1 Самки весом от 500 до 700 кг

```
db.unicorns.find(
  {
    gender: "f",
    weight: { $gte: 500, $lte: 700 }
  },
  {
    _id: 0
  }
)
```

2.3.2 Самцы, любящие grape и lemon, весом от 500 кг

```
db.unicorns.find(
```



Рис. 8: Альтернатива через find + limit

```
  {
    gender: "m",
    weight: { $gte: 500 },
    loves: { $all: ["grape", "lemon"] }
  },
  {
    _id: 0
  }
)
```

2.3.3 Единороги без поля vampires

```
db.unicorns.find(
  {
    vampires: { $exists: false }
  }
)
```

2.3.4 Список имён самцов и первое предпочтение

```
db.unicorns.find(
  { gender: "m" },
  {
    _id: 0,
    name: 1,
    loves: { $slice: 1 }
  }
).sort({ name: 1 })
```



Рис. 9: Самцы без инфы о поле и предпочтениях

3.1.1 Работа с вложенными объектами

```
// Добавление городов
db.towns.insertMany([...])

// Независимые мэры (party: "I")
db.towns.find(
  { "mayor.party": "I" },
  { _id: 0, name: 1, mayor: 1 }
)

// Беспартийные мэры (party отсутствует)
db.towns.find(
  { "mayor.party": { $exists: false } },
  { _id: 0, name: 1, mayor: 1 }
)
```



Рис. 10: Все в обратном порядке добавления(новые сверху)

3.1.2 Использование курсора и forEach

```
// Создание курсора: первые два самца по алфавиту
var cursor = db.unicorns.find(
  { gender: "m" },
  { _id: 0, name: 1 }
).sort({ name: 1 }).limit(2)

// Обход курсора с forEach
cursor.forEach(function(unicorn) {
  print(unicorn.name)
})
```

3.2.1 Количество самок с весом 500–600 кг

```
db.unicorns.countDocuments({
  gender: "f",
  weight: { $gte: 500, $lte: 600 }
})
```



Рис. 11: Все в без id с первым предпочтением

3.2.2 Список уникальных предпочтений

```
db.unicorns.distinct("loves")
```

3.2.3 Количество особей по полу

```
db.unicorns.aggregate([
  {
    $group: {
      _id: "$gender",
      count: { $sum: 1 }
    }
  }
])
```

3.3.1 Вставка Barny (аналог save)

```
db.unicorns.insertOne({
  name: "Barny",
  loves: ["grape"],
```



Рис. 12: Самки от 500 о 700 кг

```
  weight: 340,
  gender: "m"
})
db.unicorns.find({ name: "Barny" })
```

3.3.2 Обновление данных для Ayna

```
db.unicorns.updateOne(
  { name: "Ayna" },
  {
    $set: {
      weight: 800,
      vampires: 51
    }
  }
)

db.unicorns.find({ name: "Ayna" })
```

3.3.3 Обновление предпочтений Raleigh

```
db.unicorns.updateOne(
  { name: "Raleigh" },
```



Рис. 13: Самцы, любящие grape и lemon, весом от 500 кг

```
  {
    $push: { loves: "redbull" }
  }
)

db.unicorns.find({ name: "Raleigh" })
```

3.3.4 Увеличение количества вампиров у самцов

```
db.unicorns.updateMany(
  { gender: "m" },
  { $inc: { vampires: 5 } }
)

db.unicorns.find(
  { gender: "m" },
  { _id: 0, name: 1, vampires: 1 }
)
```

3.3.5 Удаление информации о партии мэра Portland

```
db.towns.updateOne(
```



Рис. 14: Единороги без поля vampires



Рис. 15: Список имён самцов и первое предпочтение

```
  { name: "Portland" },
  { $unset: { "mayor.party": 1 } }
)

db.towns.find(
  { name: "Portland" },
  { _id: 0, name: 1, mayor: 1 }
)
```



Рис. 16: Добавление городов

3.3.6 Добавление шоколада к предпочтениям Pilot

```
db.unicorns.updateOne(
  { name: "Pilot" },
  { $push: { loves: "chocolate" } }
)

db.unicorns.find(
  { name: "Pilot" },
  { _id: 0, name: 1, loves: 1 }
)
```

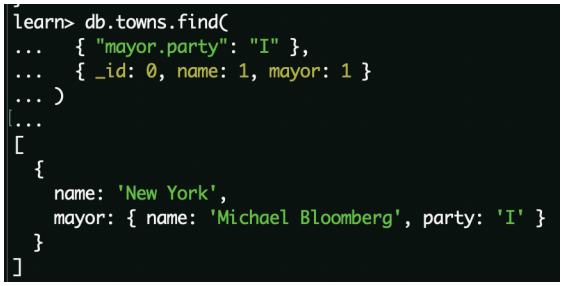


Рис. 17: Независимые мэры

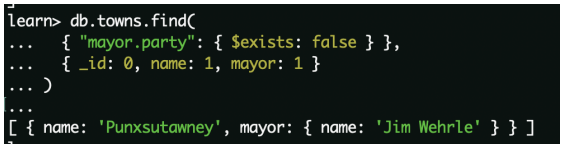


Рис. 18: Беспартийные мэры

3.3.7 Добавление sugar и lemons к предпочтениям Aurora

```
db.unicorns.updateOne(
  { name: "Aurora" },
  { $addToSet: { loves: { $each: ["sugar", "lemons"] } } }
)

db.unicorns.find(
  { name: "Aurora" },
  { _id: 0, name: 1, loves: 1 }
)
```

3.4.1 Удаление и очистка коллекции towns

3.4.1 Создание и очистка коллекции towns

```
// Вставка городов
db.towns.insertMany([
  {
    name: "Punxsutawney",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
```

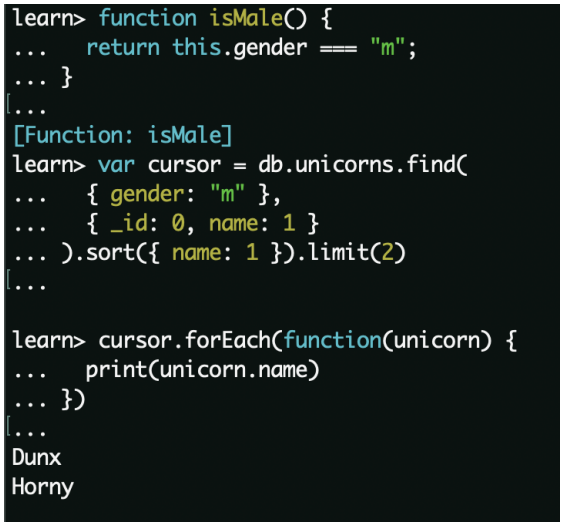


Рис. 19: Создание курсора и его обход с forEach

```
  },
  {
    name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])

// Удалить беспартийных мэров
db.towns.deleteMany({ "mayor.party": { $exists: false } })

// Проверить содержимое
db.towns.find().pretty()

// Очистить коллекцию
```



Рис. 20: Задания 3.2.1-3.2.3 + 3.3.1

```
db.towns.deleteMany({})

// Показать список коллекций
show collections
```

4.1.1 Зоны обитания и ссылки на них (DBRef)

```
// Создание зон обитания
db.habitats.insertMany([
  {
    _id: "forest",
    name: "Enchanted Forest",
    description: "Волшебный лес с сияющими растениями"
  },
  {
```



Рис. 21: Задание 3.3.2

```
    _id: "mountains",
    name: "Crystal Mountains",
    description: "Горная местность, где рождаются радуги"
  },
  {
    _id: "valley",
    name: "Golden Valley",
    description: "Уютная долина, полная мирных созданий"
  }
])

// Привязка единорогов
db.unicorns.updateOne(
  { name: "Aurora" },
  { $set: { habitat: { $ref: "habitats", $id: "forest" } } }
)
```



Рис. 22: Задание 3.3.3

```
db.unicorns.updateOne(
  { name: "Pilot" },
  { $set: { habitat: { $ref: "habitats", $id: "mountains" } } }
)

db.unicorns.updateOne(
  { name: "Leia" },
  { $set: { habitat: { $ref: "habitats", $id: "valley" } } }
)

// Проверка
db.unicorns.find(
  { habitat: { $exists: true } },
  { _id: 0, name: 1, habitat: 1 }
)
```



Рис. 23: Задание 3.3.4

4.2.1 Создание уникального индекса по полю name

```
// Попытка создать уникальный индекс
db.unicorns.createIndex(
  { name: 1 },
  { unique: true }
)

// Проверка списка индексов
db.unicorns.getIndexes()
```

4.3.1 Удаление индексов в коллекции unicorns

```
// Получение списка индексов
db.unicorns.getIndexes()

// Удаление индекса по name (если он был создан)
db.unicorns.dropIndex("name_1")
```



Рис. 24: Задание 3.3.5



Рис. 25: Задание 3.3.6

```
// Попытка удалить индекс _id (ожидается ошибка)
db.unicorns.dropIndex("_id_")
```

4.4.1 Альтернативный анализ производительности

```
// Без индекса
db.numbers.explain("executionStats").find({ value: 99999 })

// Создание индекса
db.numbers.createIndex({ value: 1 })

// С индексом
db.numbers.explain("executionStats").find({ value: 99999 })
```

Запрос с индексом выполняется быстрее, а значит он более эффективен, чем первый запрос.



Рис. 26: Задание 3.3.7

1 Вывод

В ходе лабораторной работы были освоены базовые приёмы работы с документно-ориентированной базой данных MongoDB. Были изучены основные операции: вставка, изменение и удаление документов, работа с массивами и вложенными объектами, создание и использование индексов. Также выполнен анализ производительности запросов с помощью метода ‘explain‘. Отдельное внимание уделено связыванию коллекций, а также использованию операторов ‘set‘, ‘unset‘, ‘push‘, ‘addToSet‘, ‘inc‘. Работа способствовала формированию практических навыков эффективной работы с MongoDB и пониманию принципов её внутренней логики.



Рис. 27: Задание 3.4.1 часть 1



Рис. 28: Задание 3.4.1 часть 2



Рис. 29: Задание 4.1.1 часть 1



Рис. 30: Задание 4.1.1 часть 2



Рис. 31: Задание 4.2.1

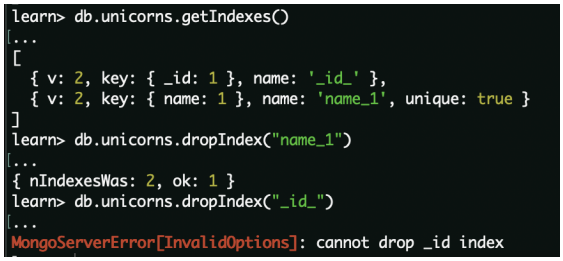


Рис. 32: Задание 4.3.1

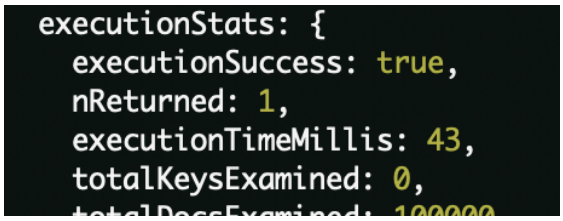


Рис. 33: Задание 4.4.1 до создания индекса

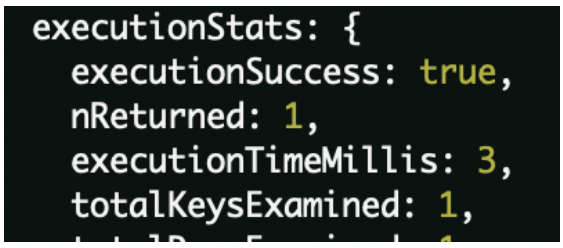


Рис. 34: Задание 4.4.1 после создания индекса